Gen AI Group Project Report (Group 27 Section B)

Project: Ledger - AI Financial Document Analyst | Assignment 12: FinTech AI

Abstract

Ledger is a generative AI based financial document analysis platform designed to reduce the time required for fundamental analysis. The application accepts annual reports, 10-K/10-Q style filings, earnings releases, and call transcripts, then converts unstructured document text into analyst-oriented outputs: extracted metrics, management tone, risk factors, competitor benchmarks, and an investment memo. The implementation combines a FastAPI backend, a React/Vite frontend, SQLite persistence, PDF/DOCX ingestion, and Google Gemini structured generation. The project is built around a practical analyst workflow: upload a filing, monitor chunked analysis progress, review the extracted data, compare companies or periods, and read a memo grounded in the extracted evidence.

Introduction

Financial statements are information dense, repetitive, and difficult to compare quickly across companies or reporting periods. Analysts must search for revenue, margins, cash flow, debt, guidance, management commentary, and changes in risk disclosures before forming an investment view. In fast-moving markets, this manual process becomes a bottleneck. Ledger addresses this by using Gen AI to read documents in sections, normalize important facts, and present results in a format that resembles an analyst workbench rather than a generic chatbot.

The application is deployed as a web app and organized into a full-stack system. The frontend guides users through uploading filings, viewing analysis dashboards, reading generated memos, and creating benchmark comparisons. The backend performs ingestion, chunking, structured LLM calls, persistence, and cross-document comparison APIs. The goal is not to replace analyst judgment, but to automate the extraction and first-pass comparison work so users can spend more time interpreting the numbers.

Problem Statement

The assignment asks for a financial document analysis platform that can ingest filings and transcripts, extract key financial metrics, compare results period over period, analyze management tone, identify risks, benchmark competitors, and generate an investment memo. The success criteria require reliable extraction of named financial figures, correct identification of cautious versus confident management language, detection of new or escalated risk factors, accurate benchmarking tables, and bull/bear memo content grounded in extracted data.

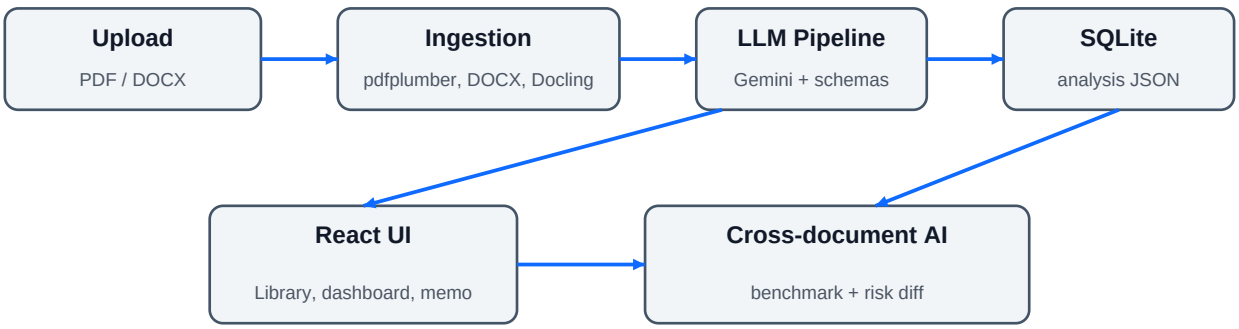
Ledger converts this requirement into a document-first workflow. Since long filings can exceed model context and output limits, the project uses chunked processing and typed schemas. Each AI output is constrained into defined Pydantic models, stored, and rendered in the UI. This keeps the system explainable and usable for repeated analysis rather than producing one-off free-form answers.

Approach

The core approach is to separate document handling, AI reasoning, and presentation into clear stages. The backend first extracts text from PDF or DOCX files. It then chunks the text so that large filings can be processed safely. Gemini is used through structured output calls, and each call returns typed data such as metrics, tone passages, risk factors, document identity, or memo sections. The final FinancialAnalysis object is stored in SQLite as JSON and later reused for dashboard rendering, benchmarking, and risk comparison.

- Document ingestion uses pdfplumber for PDFs, python-docx for DOCX files, and a Docling fallback for scanned or weak text-layer PDFs.
- Financial extraction maps reported labels to canonical metrics such as revenue, EBITDA, margins, free cash flow, capex, total debt, cash, and guidance revenue.
- Tone analysis scores sentiment, confidence, and hedging while surfacing representative confident or cautious passages.
- Risk extraction categorizes risk factors and assigns a severity score; a separate comparison call labels risks as new, escalated, unchanged, or removed.
- Benchmarking assembles the metric table in code from stored extracted values, then uses one LLM call only for comparative commentary.
- Memo generation synthesizes a company overview, financial summary, bull case, bear case, risks, and investigation questions from the structured outputs.

App Architecture



The UI polls FastAPI while the background task updates progress, then renders the stored typed analysis.

The architecture follows a client-server pattern. React handles navigation and stateful user interactions. FastAPI exposes upload, analysis, retrieval, benchmark, and risk comparison endpoints. SQLite stores uploaded document rows and the completed analysis JSON. The LLM layer is isolated behind a Gemini client and schema sanitizer, allowing the rest of the pipeline to work with typed Python models instead of raw model text.

Layer	Main files / modules	Responsibility
Frontend	Library, Analysis, Document, Memo, Benchmark pages	Upload filings, poll status, show dashboards, memos, tables, and comparisons.
API	contracts.py, compare.py	Expose upload/analyze/get/list/delete plus benchmark and risk-diff endpoints.
Pipeline	runner.py, metrics.py, tone.py, risk_factors.py, memo.py, benchmark.py	Coordinate AI stages and produce structured analysis outputs.
Ingestion	pdf.py, docx.py, router.py, chunking.py, structure.py	Parse files, chunk text, map/reduce document structure, stream progress.
LLM + Schemas	client.py, schema.py, financial.py, models.py	Generate Gemini-safe structured responses and validate typed output.
Persistence	SQLModel Contract table + SQLite	Save filename, file path, status, errors, and final analysis JSON.

Working Pipeline

The working pipeline begins in the Library page. A user selects a PDF or DOCX file, the frontend sends it to POST /contracts, and the backend saves it with status uploaded. Immediately after upload, the frontend calls POST /contracts/{id}/analyze, which starts a FastAPI background task. The user is redirected to the analysis route, where the page polls GET /contracts/{id} every two seconds. While the backend runs, an in-process progress store reports the current stage and chunk position, allowing the UI to display the live chunk progress animation.

Step	Action	Output
1	Upload filing or transcript through React Library page.	Contract row with id, filename, file path, format, status.
2	Parse file using pdfplumber, python-docx, or Docling fallback.	Full extracted text for downstream processing.
3	Structure document using chunked map-reduce.	Title, outline, and ordered document blocks.
4	Identify company, period, and document type.	DocumentIdentity model.
5	Extract metrics over chunks and deduplicate by metric and period.	Canonical metric table with reported values and source snippets.
6	Analyze tone over management commentary window.	Sentiment, confidence score, hedging level, and passages.
7	Extract risk factors over chunks and deduplicate.	Categorized risk list with severity.
8	Generate investment memo from identity, metrics, tone, and risks.	Overview, financial summary, bull case, bear case, risks, questions.
9	Persist FinancialAnalysis JSON and render UI views.	Dashboard, document outline, memo, benchmark, and risk-diff screens.

A key implementation detail is the map-reduce structure stage. Instead of asking the model to structure an entire long filing in one response, the backend splits the text, asks the model to extract ordered blocks per chunk, merges overlapping blocks in code, and then makes a bounded synthesis call over headings to produce a clean outline. Metric and risk extraction use a similar chunked strategy to improve recall. Best-effort error handling prevents one failed chunk or stage from breaking the full analysis.

Core Feature Implementation

Financial metric extraction. The backend defines canonical metrics in the financial schema so that labels like total revenue, net revenue, and reported revenue can be mapped into consistent columns. Extracted metrics include reported label, period, value, normalized numeric value where possible, unit, basis, and source text. The frontend renders these through a metrics table for analyst review.

Management tone analysis. The tone stage focuses on a large leading window of the document, which usually contains management commentary, earnings discussion, or MD&A; material. It returns an overall sentiment label, a confidence score, hedging level, a summary, and specific passages. This directly supports the requirement to distinguish cautious and confident language.

Risk factor extraction and comparison. Risk factors are extracted across chunks, categorized, given severity, and deduplicated. For period-over-period comparison, the backend sends prior and current risk lists to Gemini and receives typed deltas. The UI labels these deltas as new, escalated, unchanged, or removed.

Competitor benchmarking. Benchmarking intentionally keeps the table numeric and faithful by assembling values in code from stored analyses. The LLM adds highlights after the grid is built, which reduces the chance that model commentary changes the underlying numbers.

Investment memo generation. The memo is created after identity, metrics, tone, and risk extraction are complete. Its sections match the assignment: company overview, financial summary, bull case, bear case, key risks, and questions to investigate. This makes the final output useful for discussion rather than merely data extraction.

Discussion

The project design is pragmatic for a student Gen AI financial analysis platform. It avoids a single unbounded prompt and instead decomposes the task into smaller typed operations. This improves reliability, enables UI progress feedback, and makes cross-document features possible because every completed document has a stored structured representation. The React interface also reflects an analyst workflow: upload from the library, view the dashboard, inspect the document structure, read the memo, and compare filings from the benchmark page.

The main trade-off is that full filings can take several minutes because multiple Gemini calls are required across chunks and stages. The current architecture uses in-process background tasks, an in-process progress store, local uploads, and SQLite, which is appropriate for development and single-instance deployment. At larger scale, the same design would benefit from object storage, a durable queue, a persistent progress/event store, and a production database. The README also notes that the app is intended as a stateful single-instance deployment unless these components are replaced.

Another important point is grounding. The metric schema stores source snippets and reported values, and the benchmark table is constructed from extracted metrics rather than free-form generation. This is the right direction for financial AI, where hallucinated numbers would be unacceptable. Future improvements could include page-level citations, Excel export, SEC filing fetch by ticker, stronger numeric normalization, and regression tests against a larger financial filing benchmark.

Evaluation Against Success Metrics

Success metric	How Ledger addresses it
Named financial figures are extracted	Canonical metric schema plus chunked extraction identifies revenue, margins, EBITDA, free cash flow, debt, capex, cash, EPS, and guidance figures.
Cautious vs confident tone is identified	ToneAnalysis captures sentiment, confidence score, hedging level, rationale, and representative passages.
New risk in year 2 is flagged	Risk comparison endpoint compares prior/current extracted risks and returns typed deltas including new and escalated.
Benchmark table is accurate	Table values are assembled in code from stored extracted metrics before LLM commentary is added.
Memo contains bull and bear cases grounded in data	Memo stage consumes the extracted identity, metrics, tone, and risk factors instead of relying on the original document alone.

Conclusion

Ledger successfully implements the assignment as a usable financial document analyst. It combines document ingestion, chunked Gen AI extraction, typed schema validation, dashboard presentation, risk comparison, benchmarking, and memo generation. The most valuable engineering choice is the structured pipeline: every stage produces data the UI and later comparisons can reuse. This makes the application closer to an analyst tool than a simple prompt interface. With additional production infrastructure and larger evaluation datasets, the same architecture can be extended into a robust financial intelligence platform.

Team Contributions

Member	Student ID	Email	Contribution
Aryan Jhakar	24BCS10305	aryan.24bcs10305@sst.scaler.com	Backend + Frontend
Tanush Shoor	24BCS10265	tanush.24bcs10265@sst.scaler.com	Backend + Project Report
Parvam Shah	24BCS10231	parvam.24bcs10231@sst.scaler.com	Backend + Frontend + Architecture
Akshat Kushwaha	24BCS10060	akshat.24bcs10060@sst.scaler.com	Backend + Frontend + Architecture
Vivek Singh Solanki	24BCS10338	vivek.24bcs10338@sst.scaler.com	Backend + Frontend + README.md

Repository analyzed: AJ5831A/finance-intel. Live app referenced: finance-intel-alpha.vercel.app. The report content is based on the inspected README, backend pipeline modules, API routes, schemas, and frontend page implementation.